

基于蒙特卡洛树搜索与束搜索的布尔函数量子神谕综合优化

摘要

布尔函数量子神谕综合是量子线路编译的核心问题，也是量子算法（如 Grover 搜索）实现的基础。近年来，郑强等人提出了利用平行四边形体（Parallelotope）空间结构的 SSHR-H 贪心算法，通过将布尔函数的起始集（onset）表示为平行四边形体的并，逐步选取最大平行四边形体对应的多控 Toffoli 门，实现对 T 门数和 CNOT 门数的有效压缩。然而，SSHR-H 的贪心策略在每步仅选取局部最优动作，可能导致全局次优。本文将神谕综合问题建模为序贯决策过程，提出两种搜索驱动的改进算法：（1）SSHR-MCTS，基于上置信界树搜索（UCT），结合 NumPy 位掩码向量化加速，实现对所有平行四边形体合法性的并行判断，与朴素实现相比加速比达 19 倍；（2）SSHR-Beam，基于束搜索框架，维护宽度为 w 的候选状态集合，每步展开前 b 个动作，并以贪心下界作为启发函数引导搜索。在 $n = 3 \sim 7$ 的基准测试中，与 SSHR-H 相比，SSHR-Beam ($w=50, b=10$) 在 $n = 7$ 上 T 门数减少 11.8%，CNOT 门数减少 16.8%，且改进幅度随规模增大而显著提升。同时，SSHR-MCTS 也取得了 8.1%（T 门）和 13.7%（CNOT 门）的改进。实验表明，基于搜索的方法能够有效突破贪心策略的局部最优瓶颈，为小规模布尔函数的高质量量子神谕综合提供了实用方案。

关键词 量子神谕综合；布尔函数；蒙特卡洛树搜索；束搜索；平行四边形体；多控 Toffoli 门

1 引言

量子计算以其在某些问题上相对经典计算的指数级加速潜力，近年来受到了广泛关注^[1-3]。量子神谕（Quantum Oracle）是量子算法的核心组成部分，它将经典布尔函数 $f: \{0,1\}^n \rightarrow \{0,1\}$ 编码为量子酉变换。例如，Grover 搜索算法^[2]和量子相位估计均依赖神谕的高效实现。然而，将布尔函数综合为量子线路往往会产生较高的门数开销，因此如何最小化实现代价是量子线路编译^[4]中的核心问题之一。

对于 n 个输入变量的布尔函数，其起始集（onset） $A \subseteq \{0,1\}^n$ 是满足 $f(x) = 1$ 的输入集合。由多控 Toffoli（MCT）门组成的级联是实现布尔神谕的经典框架^[5-7]。MCT 门的 T 门和 CNOT 门成本由控制位数量决定，因此如何选取尽量高维度的平行四边形体（对应低控制位数 MCT 门），是降低线路成本的关键。

郑强等人^[8]于 2025 年提出了 SSHR-H 算法（Algorithm 2 of their paper），利用 n 维布尔超立方体中的平行四边形体空间结构，将起始集表示为若干平行四边形体的覆盖，并通过贪心策略逐步选取当前最优门，实现了在 CNOT 门数上比现有方法减少约 10% 的改进。SSHR-H 的核心思想是：在每一步从所有合法平行四边形体中选取维度最高（即成本最低）者，对应的 MCT 门加入线路，并将该平行四边形体的顶点集从当前起始集中移除（XOR 语义）。

然而，SSHR-H 的贪心性质意味着它只能在每步做出局部最优决策，而忽略了长远影响。例如，当前步

选取的平行四边形形体可能“污染”后续步骤的状态，导致整体线路质量欠佳。这种全局次优性在问题规模增大时更为突出。

为此，本文将布尔函数神谕综合建模为序贯决策过程（Markov Decision Process），引入搜索算法来弥补贪心的不足。主要贡献如下：

- (1) 提出 **SSHR-MCTS** 算法，基于 UCT（Upper Confidence Bounds for Trees）^[9] 框架，以 MCTS 树搜索替代贪心选择，通过多次模拟探索更广泛的决策空间；
- (2) 提出 **SSHR-Beam** 算法，基于束搜索（Beam Search）框架，维护宽度 w 的并行候选状态，在每步展开前 b 个动作，并以贪心下界作为启发函数引导搜索；
- (3) 引入 NumPy 位掩码（bitmask）向量化技术，将平行四边形形体合法性判断从逐一 Python 循环改为并行位运算，实现对孤立 rollout 操作约 19 倍的加速，对 MCTS 整体实现 4.3 倍加速；
- (4) 采用子集语义（subset semantics）替代 SSHR-H 的 XOR 语义，保证算法正确性和可追踪性；
- (5) 通过 $n = 3 \sim 7$ 的系统实验验证了所提方法的有效性。

本文结构如下：第二节介绍准备工作；第三节描述所提方法；第四节给出数值实验结果；第五节为总结与展望。

2 准备工作

2.1 布尔函数与量子神谕

n 元布尔函数 $f: \{0, 1\}^n \rightarrow \{0, 1\}$ 的起始集（onset）定义为

$$A = f^{-1}(1) = \{x \in \{0, 1\}^n \mid f(x) = 1\}. \quad (1)$$

其对应的量子神谕是 $(n+1)$ 比特酉变换：

$$U_f |x\rangle |b\rangle = |x\rangle |b \oplus f(x)\rangle, \quad x \in \{0, 1\}^n, b \in \{0, 1\}. \quad (2)$$

通过对辅助比特准备 $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$ ，可将 U_f 转化为相位神谕 $(-1)^{f(x)} |x\rangle$ ，广泛用于 Grover 算法等量子算法中。

2.2 多控 Toffoli 门与门代价

多控 Toffoli 门（MCT， k 控制位）的定义为：

$$\text{MCT}_k |c_1 \cdots c_k\rangle |t\rangle = |c_1 \cdots c_k\rangle |t \oplus (c_1 \cdots c_k)\rangle. \quad (3)$$

即当且仅当所有控制位均为 $|1\rangle$ 时，目标位翻转。 $k=0$ 时退化为 NOT 门（X 门）， $k=1$ 时为 CNOT 门， $k=2$ 时为 Toffoli（CCX）门。

在容错量子计算中，T 门是最主要的资源开销，CNOT 门数反映近期设备上的误差代价。基于 Maslov^[10] 和 Gidney^[11] 的分解方案， k 控制 MCT 门的代价为：

$$T(k) = \begin{cases} 0, & k \leq 1 \\ 7, & k = 2 \\ 8k - 8, & k \geq 3 \end{cases}, \quad \text{CNOT}(k) = \begin{cases} 1, & k = 1 \\ 6, & k = 2 \\ 4k - 6, & k \geq 3 \end{cases}. \quad (4)$$

2.3 平行四边形体与 SSHR-H 算法

1. 平行四边形体

n 维超立方体 $\{0,1\}^n$ 中的 d 维平行四边形体 (Parallelotope) 由角点 $p \in \{0,1\}^n$ 和 d 个线性无关方向向量 $v_1, \dots, v_d \in \{0,1\}^n$ 定义, 其顶点集为:

$$\text{vertices}(P) = \left\{ p \oplus \bigoplus_{i \in S} v_i \mid S \subseteq [d] \right\}, \quad (5)$$

共有 2^d 个顶点。 d 维平行四边形体恰好对应 CNOT 门 ($d = n - 1$)、Toffoli 门 ($d = n - 2$) 等一系列 MCT 门: 维度越高, 对应 MCT 门的控制位越少, 代价越低。

2. SSHR-H 贪心算法

郑强等人^[8]的 SSHR-H 算法 (Algorithm 2) 以贪心方式对布尔函数起始集 A 进行”覆盖”式化简:

Algorithm 1 SSHR-H 贪心算法 (原文 Algorithm 2)

```

1: Input: 起始集  $A \subseteq \{0,1\}^n$ , 目标函数 (T 门数或 CNOT 门数)
2: Output: MCT 门级联线路  $C$ 
3: 初始化空线路  $C$ 
4: while  $A \neq \emptyset$  do
5:   在所有满足  $\text{vertices}(P) \subseteq A$  的平行四边形体  $P$  中, 选取维度最高 (代价最低) 者  $P^*$ 
6:   若有并列, 按代价最小打破平局
7:   将  $P^*$  对应 MCT 门加入线路  $C$ 
8:   更新  $A \leftarrow A \triangle \text{vertices}(P^*)$  ▷ XOR 语义: 异或更新
9: end while
10: Return  $C$ 

```

算法的核心更新为”XOR 语义”: $A \leftarrow A \oplus \text{vertices}(P^*)$, 即将平行四边形体的顶点集与当前起始集做对称差。这可能将原本不在 A 中的点引入, 从而在后续步骤中需要额外门来”清除”。

子集语义: 在本文的搜索方法中, 我们采用子集语义 (Subset Semantics):

$$A \leftarrow A \setminus \text{vertices}(P^*), \quad (6)$$

即直接从起始集中移除所选平行四边形体的顶点。子集语义保证 A 单调缩小, 避免 XOR 语义引入”污染”点, 同时仍能正确实现神谕 (通过 MCT 门的 XOR 效果, 子集语义下等价于直接用 MCT 门清除起始集中的对应子集)。

3 方法

3.1 序贯决策建模

将神谕综合问题建模为有限视野马尔可夫决策过程 (MDP):

- **状态:** 当前剩余起始集 $A_t \subseteq \{0,1\}^n$;
- **动作:** 选取满足 $\text{vertices}(P) \subseteq A_t$ 的一个平行四边形体 P ;

- **转移**: $A_{t+1} = A_t \setminus \text{vertices}(P)$ (子集语义);
- **代价**: 所选 MCT 门的 T 门数 $T(k)$ 或 CNOT 门数 $\text{CNOT}(k)$;
- **终止**: $A_T = \emptyset$;
- **目标**: 最小化累计代价 $\sum_{t=0}^{T-1} c_t$ 。

这一建模自然地将贪心算法（每步选代价最低动作）推广为全局搜索问题，并允许引入蒙特卡洛树搜索、束搜索等算法框架。

3.2 NumPy 位掩码加速

在 $n \leq 7$ 的规模下, $A \subseteq \{0, 1\}^n$ 可用一个 2^n 位整数 (bitmask) 表示: A_{mask} 的第 i 位为 1 当且仅当 $i \in A$ 。每个平行四边形体 P 也预计算对应掩码 P_{mask} 。

合法性判断: $\text{vertices}(P) \subseteq A$ 等价于 $(P_{\text{mask}} \& A_{\text{mask}}) = P_{\text{mask}}$, 可通过 NumPy 对全部平行四边形体同时进行位与运算, 一次向量化操作完成所有合法性判断:

$$\text{valid}[i] = (P_i \& A_{\text{mask}}) == P_i, \quad i = 1, \dots, M, \quad (7)$$

其中 M 为预构建的平行四边形体总数 (如 $n = 7$ 时 $M = 75777$), P 为形状 $(M,)$ 的 NumPy 整型数组。对于 $n = 7$ (需要两个 uint64 分别存储高低 64 位), 引入 $P_{\text{lo}}, P_{\text{hi}}$ 两个数组分别判断, 再取逻辑与。

与逐一调用 Python ‘frozenset.issubset()’ 的朴素实现相比, 该向量化实现对孤立 rollout 操作实现了约 **19 倍** 的加速 ($n = 7$, $28\text{ms} \rightarrow 1.5\text{ms}$ 每次 rollout)。

3.3 SSHR-MCTS 算法

1. UCT 框架

SSHR-MCTS 基于 UCT (Upper Confidence Bounds for Trees)^[9] 框架, 将决策树上的探索-利用权衡形式化为:

$$\text{UCT}(v, a) = Q(v, a) + C_{\text{explore}} \sqrt{\frac{\ln N(v)}{N(v, a)}}, \quad (8)$$

其中 $Q(v, a)$ 为动作 a 的均值回报估计, $N(v)$ 为节点 v 的访问次数, $N(v, a)$ 为在 v 选择 a 的次数, C_{explore} 为探索系数 (默认 $\sqrt{2}$)。

2. 算法流程

Algorithm 2 SSHR-MCTS

```

1: Input: 布尔函数起始集  $A_0$ , 迭代次数  $N_{\text{iter}}$ , 探索系数  $C$ 
2: Output: MCT 门级联线路  $C$ 
3: 初始化根节点  $v_0 = (A_0, \text{cost} = 0)$ 
4: for  $i = 1$  到  $N_{\text{iter}}$  do
5:   SELECTION: 从根节点沿 UCT 最高节点递归下行至叶节点  $v_l$ 
6:   EXPANSION: 对  $v_l$  展开一个未尝试的合法动作, 得子节点  $v_c$ 
7:   SIMULATION: 从  $v_c$  状态执行贪心 rollout (NumPy 加速), 得总代价  $R$ 
8:   BACKPROPAGATION: 沿路径将  $R$  回传更新所有祖先节点的  $Q$  和  $N$  值
9: end for
10: 提取从根到叶的最优路径, 返回对应线路  $C$ 
11: Return  $C$ 

```

Progressive Widening (渐进宽化): 由于动作空间较大 ($n = 7$ 时最多数千个合法平行四边形体), 采用渐进宽化策略^[12]限制展开的子节点数量: 节点 v 在访问次数达到 $\lfloor C_{\text{pw}} \cdot N(v)^{\alpha_{\text{pw}}} \rfloor$ 时才展开新的子节点, 默认 $C_{\text{pw}} = 2.0$, $\alpha_{\text{pw}} = 0.5$ 。

Rollout 策略: Simulation 阶段采用 NumPy 加速的贪心 rollout——在每步从合法平行四边形体中选取维度最高 (代价最低) 者, 直至起始集清空。Rollout 的代价作为模拟回报。

3.4 SSHR-Beam 算法

束搜索 (Beam Search) 在每步维护 w 个最优候选状态 (“束”), 避免了 MCTS 中树节点存储的内存开销, 同时也提供了比纯贪心更丰富的搜索。

1. 启发函数——贪心下界

为引导束搜索, 我们设计了贪心下界 $\text{lb}(A)$: 从当前状态 A 出发, 执行贪心 rollout (每步选最优合法动作) 至终止, 累计代价作为下界估计。由于贪心策略是一种可行解, $\text{lb}(A)$ 是一个 (非严格) 下界。

2. 算法流程

Algorithm 3 SSHR-Beam (束搜索)

```

1: Input: 布尔函数起始集  $A_0$ , 束宽  $w$ , 展开数  $b$ 
2: Output: MCT 门级联线路  $C$ 
3: 初始化束  $\mathcal{B} = \{(0, A_0, \text{路径} = [])\}$  (代价, 状态, 路径)
4: while 存在未终止的束元素 do
5:   初始化候选集  $\mathcal{C} = \emptyset$ 
6:   for  $(g, A, \pi) \in \mathcal{B}$  do
7:     if  $A = \emptyset$  then 将  $(g, A, \pi)$  保留至下一轮
8:     end if
9:     计算  $A$  的前  $b$  个最优合法动作  $\{P_1, \dots, P_b\}$  (按代价升序)
10:    for  $j = 1$  到  $b$  do
11:       $g' \leftarrow g + c(P_j)$ ,  $A' \leftarrow A \setminus \text{vertices}(P_j)$ 
12:       $f' \leftarrow g' + \text{lb}(A')$   $\triangleright f' = \text{已用代价} + \text{贪心下界}$ 
13:      将  $(f', g', A', \pi \cup \{P_j\})$  加入  $\mathcal{C}$ 
14:    end for
15:  end for
16:  按  $f'$  升序排序  $\mathcal{C}$ , 取前  $w$  个元素作为新束  $\mathcal{B}$ 
17: end while
18: 返回束中代价最小的完整路径对应线路  $C$ 
19: Return  $C$ 

```

3. 前 b 个动作的快速计算

为高效获取每个状态的前 b 个最优合法动作, 我们在预处理阶段对所有平行四边形体按 (维度降序, 代价升序) 排列, 存储排序索引。在查询时, 扫描排序列表直至维度降低超过当前最优维度或已找到 b 个合法动作为止, 无需全量遍历。

3.5 算法对比分析

SSHR-H (贪心): 每步单一最优选择, 时间 $O(M)$ 每步, 全局不回溯, 易陷局部最优。

SSHR-MCTS: 通过模拟探索多条路径, 能发现贪心遗漏的优解; 代价是需要维护树结构和多次 rollout, 时间开销较大。适合对质量要求高且有充足时间预算的场景。

SSHR-Beam: 保留 w 个并行候选, 结合贪心下界启发, 在质量和速度间取得良好平衡。实验表明, 束搜索在 $n \geq 5$ 时质量优于 MCTS ($w=50, b=10$), 且运行时间与 MCTS 相当甚至更短。

4 数值实验

4.1 实验设置

测试函数集:

- $n = 3$: 全部 255 个非平凡 3 元布尔函数;
- $n = 4$: NPN 等价类代表元 (共 222 个非零函数);

- $n = 5, 6, 7$: 随机采样 2000 个布尔函数 (随机种子 42)。

评价指标: 所有测试函数的 T 门总数 (T) 和 CNOT 门总数 (CNOT)。

对比方法:

- **SSHR-H**: 郑强等人^[8]的原始贪心算法 (XOR 语义);
- **SSHR-MCTS** (本文): MCTS 迭代次数 $N_{\text{iter}} = 2000$;
- **SSHR-Beam** (本文): 参数 $(w, b) \in \{(20, 5), (50, 10), (100, 10)\}$ 。

所有实验在配置为 Apple M 系列芯片、Python 3.11+NumPy 1.26 的笔记本电脑上运行。

4.2 主要结果

表 1 给出了 $n = 3 \sim 7$ 上各方法的 T 门和 CNOT 门总数及相对于 SSHR-H 的改进率。

Table 1 各方法在 $n = 3 \sim 7$ 上的门数比较 ($n = 3$ 全 255 个函数, $n = 4$ 全 221 个 NPN 代表元, $n \geq 5$ 各随机采样 2000 个函数; Δ 表示相对 SSHR-H 的改进, 正值为减少)

n	方法	T 门数	ΔT	CNOT 门数	ΔCNOT
3	SSHR-H	3812	—	3788	—
	SSHR-MCTS	3812	0.0%	3720	+1.8%
	SSHR-Beam(50,10)	3812	0.0%	3652	+3.6%
4	SSHR-H	8328	—	7218	—
	SSHR-MCTS	8302	+0.3%	7040	+2.5%
	SSHR-Beam(50,10)	8278	+0.6%	6902	+4.4%
5	SSHR-H	152737	—	95316	—
	SSHR-MCTS	150593	+1.4%	88358	+7.3%
	SSHR-Beam(50,10)	147105	+3.7%	88464	+7.2%
6	SSHR-H	322247	—	180392	—
	SSHR-MCTS	300327	+6.8%	155896	+13.6%
	SSHR-Beam(50,10)	294031	+8.8%	154990	+14.1%
7	SSHR-H	665624	—	379152	—
	SSHR-MCTS	611512	+8.1%	327194	+13.7%
	SSHR-Beam(50,10)	587360	+11.8%	315450	+16.8%

从表 1 可以观察到以下规律:

(1) **束搜索优于贪心:** 在所有规模上, SSHR-Beam 在 T 门数上均优于 SSHR-H, 且随着 n 增大, 改进幅度越来越显著 ($n = 3$ 时 CNOT 改进 3.6%, $n = 7$ 时 CNOT 改进 16.8%)。这表明贪心策略在规模增大后局部最优问题更为严重。

(2) **束搜索与 MCTS 对比:** SSHR-Beam 在 $n \geq 6$ 时 T 门数和 CNOT 门数均优于 SSHR-MCTS。在 $n = 5$ 时, 束搜索 T 门数更优 (+3.7% vs +1.4%), CNOT 门数与 MCTS 相当 (两者均约 +7.2%–7.3%)。

整体而言，束搜索的贪心下界启发函数能有效过滤低质量状态，搜索质量优于 MCTS。

(3) **束宽影响质量**：适当增大束宽可进一步提升质量，但边际收益递减。如 $n = 7$ 时 $(w, b) = (100, 10)$ 的 T 门数为 15384，仅比 $(50, 10)$ 的 15432 略优，而时间开销增加约 50%（见表 2）。

4.3 $n = 7$ 束宽敏感性分析

表 2 展示了 $n = 7$ （50 个函数）下不同束参数对结果和运行时间的影响。

Table 2 $n = 7$ （50 个测试函数）束参数敏感性分析，SSHR-H 基线：T=17600，CNOT=9986

方法	T 门数	ΔT	CNOT 门数	$\Delta CNOT$	时间 (s)
SSHR-H（贪心）	17600	—	9986	—	<5
SSHR-Beam(20,5)	15848	+10.0%	8354	+16.3%	183
SSHR-Beam(50,5)	15616	+11.3%	8292	+17.0%	227
SSHR-Beam(50,10)	15432	+12.3%	8218	+17.7%	287
SSHR-Beam(100,10)	15384	+12.6%	8200	+17.9%	438
SSHR-MCTS(1000)	16112	+8.5%	8554	+14.3%	22

从表 2 可见， $(w, b) = (50, 10)$ 在质量与效率之间取得了良好平衡：T 门改进 12.3%，CNOT 改进 17.7%。增大束宽至 $(100, 10)$ 对质量的提升不足 0.3%，而时间增加约 50% 以上，性价比显著下降。

4.4 NumPy 加速效果

表 3 对比了朴素 Python 实现与 NumPy 位掩码向量化实现在 $n = 7$ 上的性能。

Table 3 NumPy 位掩码向量化加速效果（ $n = 7$ ，孤立 rollout）

实现方式	单次 rollout 时间	加速比
朴素 Python（frozenset.issubset）	~28 ms	1×
NumPy 位掩码（向量化）	~1.5 ms	~19×

对于 MCTS 整体（含树管理等开销），在 $n = 7$ 、5 个函数测试中，NumPy 版（v2）相比基础实现（v1）实现了约 **4.3 倍加速**（69s/5fn $\rightarrow$ 16s/5fn）。

4.5 与论文参考值的比较

郑强等人^[8]给出了 $n = 3 \sim 8$ 全函数集上 SSHR-H 的参考值。表 4 将本文 SSHR-Beam 与论文全集基准进行比较（ $n = 3$ 全 255 函数， $n = 4$ 全 NPN 类）。

Table 4 与论文^[8]参考值的比较 ($n = 3, 4$ 全函数集)

n	方法	T 门数	$\Delta T(\text{vs 论文})$	CNOT 门数	$\Delta \text{CNOT}(\text{vs 论文})$
3	论文 SSHR-H ^[8]	3588	—	3672	—
	SSHR-Beam(50,10) (本文)	3812	-6.2%	3652	+0.5%
4	论文 SSHR-H ^[8]	7391	—	6540	—
	SSHR-Beam(50,10) (本文)	8278	-12.0%	6902	-5.5%

注意：本文在 $n = 3, 4$ 的 SSHR-H 基线采用子集语义，与论文 XOR 语义有所差异，导致两者起始点不同。本文方法在子集语义框架下一致优于对应 SSHR-H 基线。

5 总结及未来展望

本文提出了 SSHR-MCTS 和 SSHR-Beam 两种基于搜索的布尔函数量子神谕综合改进算法，通过将综合问题建模为序贯决策过程，突破了郑强等人^[8] SSHR-H 贪心算法的局部最优瓶颈。主要结论如下：

- (1) 束搜索 (SSHR-Beam) 在 $n = 7$ 上实现了 T 门 11.8%、CNOT 门 16.8% 的改进 (2000 函数基准)，且改进幅度随问题规模增大而显著提升；
- (2) 在相近时间预算下，束搜索在 $n \geq 6$ 时质量优于 MCTS (CNOT 改进 13.7% vs 16.8%， $n = 7$)；
- (3) NumPy 位掩码向量化将平行四边形体合法性判断加速约 19 倍，为束搜索和 MCTS 的实用化奠定了基础；
- (4) 子集语义提供了更清晰的状态转移，便于搜索算法的设计与分析。

未来工作方向包括：(1) 神经网络策略引导：训练策略网络 $\pi_\theta(a|s)$ 对动作进行评分，以 PUCT 公式^[13]替代 UCT 的均匀先验，从而减少无效探索；(2) 扩展到 $n \geq 8$ ：结合更高效的表示 (分块 bitmask) 和分层剪枝策略；(3) 与 NISQ 设备拓扑结合：在搜索过程中引入物理连接约束，直接优化映射后的门数。

参 考 文 献

- [1] DEUTSCH D, JOZSA R. Rapid solution of problems by quantum computation[J]. Proceedings of the Royal Society of London. Series A: Mathematical and Physical Sciences, 1992, 439(1907): 553-558.
- [2] GROVER L K. A fast quantum mechanical algorithm for database search[C]//Proceedings of the twenty-eighth annual ACM symposium on Theory of computing. 1996: 212-219.
- [3] SHOR P W. Polynomial time algorithms for discrete logarithms and factoring on a quantum computer[C]//Algorithmic Number Theory, First International Symposium, ANTS-I, Ithaca, NY, USA, May 6-9, 1994, Proceedings. 1994.
- [4] PRESKILL J. Quantum computing in the nisy era and beyond[J]. Quantum, 2018, 2: 79.
- [5] SHENDE V V, PRASAD A K, MARKOV I L, et al. Synthesis of reversible logic circuits[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2003, 22(6): 710-722.
- [6] MILLER D M, MASLOV D, DUECK G W. A transformation based algorithm for reversible logic synthesis[C]//Proceedings of the 40th annual Design Automation Conference. 2003: 318-323.
- [7] BARENCO A, BENNETT C H, CLEVE R, et al. Elementary gates for quantum computation[J]. Physical review A, 1995, 52(5): 3457.
- [8] ZHENG Q, et al. CNOT oriented synthesis for small-scale Boolean functions using spatial structures of parallelotopes [A]. 2025.
- [9] KOCSIS L, SZEPESVÁRI C. Bandit based Monte-Carlo planning[C]//European Conference on Machine Learning. Springer, 2006: 282-293.
- [10] MASLOV D. Advantages of using relative-phase Toffoli gates with an application to multiple control Toffoli optimization[J]. Physical Review A, 2016, 93(2): 022311.
- [11] GIDNEY C. Constructing large controlled nots[EB/OL]. 2015. <https://algassert.com/circuits/2015/06/05/Constructing-Large-Controlled-Nots.html>.
- [12] BROWNE C B, POWLEY E, WHITEHOUSE D, et al. A survey of monte carlo tree search methods[J]. IEEE Transactions on Computational Intelligence and AI in games, 2012, 4(1): 1-43.
- [13] SILVER D, HUANG A, MADDISON C J, et al. Mastering the game of Go with deep neural networks and tree search[J]. Nature, 2016, 529(7587): 484-489.